RAG & Agentic AI Flashcards

Stack: Python, Snowflake (Cortex), dbt, LangGraph, AWS. Each card has a **Question**, **Answer**, **Code**, and **Try it** tip. Your list repeated "How do you manage state in LangGraph?" twice, so it appears once (Card 24).

0. Setup guide (do this once)

Python env

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install langgraph langchain-core langchain-anthropic anthropic \
  snowflake-connector-python snowflake-snowpark-python dbt-snowflake \
  sentence-transformers faiss-cpu chromadb tiktoken rank-bm25 \
  tenacity boto3 ragas
export ANTHROPIC_API_KEY=...   # get from console.anthropic.com
```

Snowflake (free): sign up for a 30-day trial at signup.snowflake.com. Then run:

```
CREATE WAREHOUSE IF NOT EXISTS RAG_WH WAREHOUSE_SIZE='XSMALL'
AUTO_SUSPEND=60;
CREATE DATABASE IF NOT EXISTS RAG_DB;
CREATE SCHEMA IF NOT EXISTS RAG_DB.KB;
```

Cortex functions (EMBED_TEXT_768, COMPLETE, Cortex Search) are region dependent. Check the Snowflake docs for your region and enable cross-region inference if needed.

dbt: `dbt init rag_project`, then fill `~/.dbt/profiles.yml` (type: snowflake, account, user, password or key, role, database RAG_DB, warehouse RAG_WH, schema KB). Verify with `dbt debug`. Run models with `dbt build`.

No Snowflake yet? Every Python card below also works locally with `sentence-transformers` plus FAISS or Chroma. Swap in Snowflake later.

Shared helpers used in later cards

```
from sentence_transformers import SentenceTransformer
import numpy as np
_model = SentenceTransformer("all-MiniLM-L6-v2")   # 384-dim, runs on CPU
def embed(texts):
    return _model.encode(texts, normalize_embeddings=True)  # unit vectors
```

Part A: RAG fundamentals

Card 1. What is RAG, and why do enterprise AI apps need it?

Answer: Retrieval-Augmented Generation fetches relevant passages from your own data at query time and gives them to the LLM as context. Enterprises need it because LLMs (1) don't know private or fresh data, (2) hallucinate without evidence, (3) can't cite sources, and (4) retraining or fine-tuning for every data change is slow and costly. RAG also lets you enforce access control per document and update knowledge by re-indexing instead of retraining.

Code

```
import anthropic
client = anthropic.Anthropic()

def rag_answer(question, retriever):
    chunks = retriever(question, k=5)
    context = "\n\n".join(f"[{i+1}] {c['text']}" for i, c in
enumerate(chunks))
    msg = client.messages.create(
        model="claude-sonnet-5-5", max_tokens=600,
        system="Answer ONLY from the context. Cite sources like [1]. If not
found, say you don't know.",
        messages=[{"role": "user", "content":
f"Context:\n{context}\n\nQuestion: {question}"}],
    )
    return msg.content[0].text
```

Try it: Build `retriever` with the FAISS code in Card 7, then call `rag_answer`.

Card 2. Explain the complete RAG ingestion pipeline.

Answer: Source connectors, then parse and clean, then chunk, then enrich with metadata, then embed, then upsert into a vector store (plus a keyword index), then validate and monitor. Production versions are incremental (hash-based change detection), idempotent (same chunk id on re-run), and handle deletes and permission changes. In a Snowflake and dbt stack: land raw docs in a table, use dbt to clean and chunk into a `chunks` model, embed with Cortex, and keep the result in a `VECTOR` column.

Code

```
import hashlib, snowflake.connector

def chunk_id(doc_id, idx, text):
    return hashlib.sha256(f"{doc_id}:{idx}:{text}".encode()).hexdigest()[:16]

def ingest(doc, chunker, conn):
    cur = conn.cursor()
    for i, text in enumerate(chunker(doc["text"])):
        cid = chunk_id(doc["id"], i, text)
        cur.execute("""
          MERGE INTO RAG_DB.KB.CHUNKS t
          USING (SELECT %s cid, %s doc_id, %s idx, %s txt, %s src, %s ver) s
          ON t.chunk_id = s.cid
          WHEN NOT MATCHED THEN INSERT (chunk_id, doc_id, chunk_idx, text,
source, version, embedding)
            VALUES (s.cid, s.doc_id, s.idx, s.txt, s.src, s.ver,
                    SNOWFLAKE.CORTEX.EMBED_TEXT_768('snowflake-arctic-embed-
m', s.txt))
        """, (cid, doc["id"], i, text, doc["source"], doc["version"]))
```

```
-- dbt model: models/chunks_raw.sql (cleaning step)
select doc_id, regexp_replace(body, '\\s+', ' ') as clean_text, source,
version
from {{ source('raw', 'documents') }}
where body is not null
```

Try it: Create `RAG_DB.KB.CHUNKS` with a `VECTOR(FLOAT,768)` column, load 5 PDFs or markdown files, run `ingest`.

Card 3. What is a chunk, and how do you select chunk size?

Answer: A chunk is the unit of text you embed and retrieve. Too small loses context; too large dilutes the embedding and wastes tokens. Start with 300 to 500 tokens, 10 to 15% overlap, split on structure (headings, paragraphs) first. Then **tune empirically**: try 200/400/800 tokens and pick the best Recall@k and answer quality on your own eval set. FAQs favor small chunks; legal or technical docs favor larger ones or parent-child chunking (retrieve small, feed the parent section).

Code

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base")

def chunk_tokens(text, size=400, overlap=60):
    ids = enc.encode(text)
    step = size - overlap
    return [enc.decode(ids[i:i+size]) for i in range(0, max(len(ids)-overlap,
1), step)]

# tuning loop
for size in (200, 400, 800):
    chunks = chunk_tokens(corpus, size)
    # rebuild index, then compute recall_at_k(...) from Card 13
```

Try it: `pip install langchain-text-splitters` gives `RecursiveCharacterTextSplitter` for structure-aware splitting.

Card 4. What metadata would you store for every chunk?

Answer: `chunk_id`, `doc_id`, `chunk_index`, source system and URI, title and section heading, page number, author, created and updated timestamps, document version, content hash, language, **access-control tags** (tenant, role, ACL), doc type, token count, embedding model name and version, and ingestion run id. Metadata enables filtering, citations, freshness, security trimming, debugging, and safe re-embedding.

Code

```
CREATE TABLE RAG_DB.KB.CHUNKS (
  chunk_id STRING PRIMARY KEY, doc_id STRING, chunk_idx INT,
  text STRING, source STRING, uri STRING, title STRING, section STRING,
  page INT, author STRING, updated_at TIMESTAMP_NTZ, version STRING,
  content_hash STRING, language STRING, tenant_id STRING,
  allowed_roles ARRAY, token_count INT,
  embed_model STRING, ingest_run_id STRING,
  embedding VECTOR(FLOAT, 768)
);
```

```
# filtered retrieval (security trimming)
sql = """SELECT text, uri FROM RAG_DB.KB.CHUNKS
 WHERE tenant_id=%s AND ARRAY_CONTAINS(%s::VARIANT, allowed_roles)
 ORDER BY VECTOR_COSINE_SIMILARITY(embedding, %s::VECTOR(FLOAT,768)) DESC
LIMIT 5"""
```

Card 5. What happens when the question isn't answered by the knowledge base?

Answer: A well-designed system must **abstain**, not guess. Detect it via (1) a similarity or reranker score threshold, (2) an LLM relevance grader, and (3) a prompt rule: "If the context doesn't contain the answer, say so." Then fall back: ask a clarifying question, route to a human, or (if allowed) a web search tool, clearly labelled as outside the KB. Log these misses; they reveal content gaps.

Code

```
THRESHOLD = 0.35

def answer_or_abstain(q, retriever):
    hits = retriever(q, k=5)            # each hit has a 'score' in [0,1]
    if not hits or hits[0]["score"] < THRESHOLD:
        log_unanswered(q)               # feeds a content-gap dashboard
        return "I couldn't find this in the knowledge base. Want me to
connect you with a specialist?"
    return rag_answer(q, lambda *_a, **_k: hits)
```

Try it: Calibrate `THRESHOLD` by plotting scores for answerable vs unanswerable test questions.

Card 6. Explain embeddings and semantic search.

Answer: An embedding model maps text to a dense vector so that texts with similar **meaning** land close together. Semantic search embeds the query, then returns chunks whose vectors have the highest cosine similarity (or dot product, or lowest L2 distance). It matches "reset my password" with "account recovery steps" even without shared words, which keyword search misses. Use hybrid search (BM25 plus vectors) for exact terms like IDs and acronyms.

Code

```
docs = ["How to recover your account", "Quarterly revenue report", "Steps to
reset a password"]
D = embed(docs)
q = embed(["I forgot my password"])[0]
scores = D @ q                        # cosine, since vectors are normalized
for s, d in sorted(zip(scores, docs), reverse=True):
    print(round(float(s), 3), d)
```

```
-- Snowflake-native version
SELECT text FROM RAG_DB.KB.CHUNKS
ORDER BY VECTOR_COSINE_SIMILARITY(
  embedding, SNOWFLAKE.CORTEX.EMBED_TEXT_768('snowflake-arctic-embed-m', 'I
forgot my password')) DESC
LIMIT 5;
```

Card 7. How does a vector database work?

Answer: It stores vectors plus metadata and builds an **approximate nearest neighbor (ANN)** index so search doesn't compare against every vector. Common indexes: HNSW (graph, fast, high recall, memory heavy), IVF (cluster then search a few clusters), and PQ (compress vectors). You trade a little recall for big speed gains. Features to look for: metadata filtering, hybrid search, upserts and deletes, and sharding. Options: FAISS and Chroma (local), pgvector, OpenSearch, Pinecone, and Snowflake's `VECTOR` type or Cortex Search.

Code

```
import faiss, numpy as np
chunks = ["refund policy ...", "shipping times ...", "warranty terms ..."]
X = np.array(embed(chunks), dtype="float32")
index = faiss.IndexHNSWFlat(X.shape[1], 32, faiss.METRIC_INNER_PRODUCT)  #
HNSW graph
index.add(X)

def retriever(q, k=5):
    D, I = index.search(np.array(embed([q]), dtype="float32"), k)
    return [{"text": chunks[i], "score": float(s)} for s, i in zip(D[0],
I[0]) if i != -1]
```

Try it: Chroma is simpler with persistence: `chromadb.PersistentClient("./db").create_collection("kb")`.

Card 8. Why retrieve the top 5 chunks? Is five always optimal?

Answer: No. Five is a common default balancing recall (enough chances to include the answer) against noise, cost, and the "lost in the middle" effect. The best k depends on chunk size, question type (multi-hop needs more), and reranking. Common pattern: retrieve 20 to 50 candidates, rerank, and pass only the top 3 to 8. Choose k from data, with a recall-vs-tokens curve, or use adaptive k (score cutoff or elbow).

Code

```
def adaptive_k(hits, min_k=2, max_k=8, drop=0.15):
    keep = [hits[0]]
    for prev, cur in zip(hits, hits[1:]):
        if len(keep) >= max_k or (len(keep) >= min_k and prev["score"] -
cur["score"] > drop):
            break
        keep.append(cur)
    return keep

for k in (3, 5, 8, 12):   # compare using Card 13's recall_at_k + token cost
    print(k, recall_at_k(eval_set, retriever, k))
```

Card 9. Explain reranking, semantic caching, and context compression.

Answer:

- **Reranking:** a cross-encoder reads (query, chunk) *together* and scores relevance far more accurately than vector similarity. Used on the top 20 to 50 candidates.
- **Semantic caching:** if a new question is semantically near a previously answered one (similarity above ~0.95), return the cached answer. This saves LLM cost and latency. Scope caches by tenant and permissions, and expire them when documents change.
- **Context compression:** shrink retrieved text before the LLM sees it by extracting only relevant sentences or summarizing, cutting tokens while keeping evidence.

Code

```
# Reranking
from sentence_transformers import CrossEncoder
ce = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
def rerank(q, hits, top=5):
    s = ce.predict([(q, h["text"]) for h in hits])
    return [h for _, h in sorted(zip(s, hits), key=lambda x: -x[0])][:top]

# Semantic cache
cache = []   # [(vector, answer)]
def cached(q, sim=0.95):
    v = embed([q])[0]
    for cv, ans in cache:
        if float(cv @ v) >= sim: return ans
    return None

# Compression: keep only sentences similar to the query
import re
def compress(q, text, keep=3):
```

```
    sents = re.split(r"(?<=[.!?])\s+", text)
    sc = embed(sents) @ embed([q])[0]
    top = sorted(np.argsort(-sc)[:keep])
    return " ".join(sents[i] for i in top)
```

Card 10. How do you reduce hallucinations in a production RAG app?

Answer: Layer the defenses: (1) high-quality retrieval (hybrid search, reranking, good chunking); (2) a strict grounding prompt with mandatory citations; (3) abstention thresholds; (4) low temperature; (5) a **verification step** (faithfulness check that each claim is supported by context); (6) structured output with quotes or source ids; (7) continuous evaluation (RAGAS faithfulness, human review, user feedback); (8) guardrails and fallbacks for low-confidence answers.

Code

```
def verify_grounded(answer, context):
    r = client.messages.create(
        model="claude-haiku-5-5", max_tokens=200, temperature=0,
        system='You are a fact-checker. Reply JSON: {"supported": true|false,
"unsupported_claims": []}',
        messages=[{"role": "user", "content":
f"Context:\n{context}\n\nAnswer:\n{answer}"}],
    )
    import json; return json.loads(r.content[0].text)

draft = rag_answer(q, retriever)
check = verify_grounded(draft, context_text)
final = draft if check["supported"] else "I'm not confident enough to answer
from the available documents."
```

Card 11. How do you reduce token usage without reducing answer quality?

Answer: Retrieve wide, then rerank and pass fewer chunks; compress context; deduplicate near-identical chunks; use a short, tight system prompt; route easy questions to a smaller model; cache (semantic cache and provider prompt caching for static prefixes); cap output length; and summarize conversation history instead of resending it. Always measure quality (Recall, faithfulness) before and after each change.

Code

```
def dedupe(hits, thresh=0.92):
    kept, vecs = [], []
    for h in hits:
        v = embed([h["text"]])[0]
        if all(float(v @ u) < thresh for u in vecs):
            kept.append(h); vecs.append(v)
    return kept

def tokens(s): return len(enc.encode(s))
before = tokens("\n".join(h["text"] for h in hits))
after = tokens("\n".join(compress(q, h["text"]) for h in dedupe(rerank(q,
hits, 4))))
print(f"tokens {before} -> {after}")
```

Card 12. What is the difference between an LLM, an embedding model, and a reranker?

Answer:

	Input / Output	Role	Speed and cost
LLM	prompt in, generated text out	reasoning, writing the answer	slowest, most expensive
Embedding model	text in, one vector out (bi-encoder)	indexing and first-pass retrieval	fast, cheap, precomputable
Reranker	(query, doc) in, relevance score out (cross-encoder)	precise reordering of candidates	medium; run on top 20 to 50 only

Code

```
vec = embed(["refund policy"])[0]                          # embedding model
-> vector
score = ce.predict([("refund policy", "Refunds within 30 days")])[0]   #
reranker -> score
text = client.messages.create(model="claude-sonnet-5-5", max_tokens=100,
        messages=[{"role": "user", "content": "Summarize our refund
policy"}]).content[0].text  # LLM
```

Part B: Evaluation and multi-agent systems

Card 13. What is Recall@5, and how do you evaluate your top five chunks?

Answer: Recall@5 is the fraction of test questions for which at least one *relevant* chunk appears in the top 5 results: `hits / total questions`. It measures the retriever alone. Build a labeled eval set (question, ids of chunks that contain the answer), 50 to 200 questions from real user queries. Pair it with **MRR** (rank of first relevant hit), **Precision@5** (how much of the top 5 is useful), and generation metrics (faithfulness, answer relevance via RAGAS). Evaluate after every chunking, embedding, or reranker change.

Code

```
eval_set = [
    {"q": "What is the refund window?", "relevant_ids": {"c12"}},
    {"q": "How do I reset SSO?",         "relevant_ids": {"c40", "c41"}},
]

def recall_at_k(eval_set, retriever, k=5):
    hits = 0
    for ex in eval_set:
        got = {h["chunk_id"] for h in retriever(ex["q"], k=k)}
        hits += bool(got & ex["relevant_ids"])
    return hits / len(eval_set)

def mrr(eval_set, retriever, k=10):
    total = 0
    for ex in eval_set:
        for rank, h in enumerate(retriever(ex["q"], k=k), 1):
            if h["chunk_id"] in ex["relevant_ids"]:
                total += 1 / rank; break
    return total / len(eval_set)

print("Recall@5:", recall_at_k(eval_set, retriever, 5))
```

Try it: Generate a starter eval set by asking an LLM to write a question for each of 100 random chunks, then review it by hand.

Card 14. What is a multi-agent orchestration system?

Answer: A system where several specialized agents (each with its own prompt, tools, and sometimes model) collaborate on a task, coordinated by an orchestrator: a supervisor that routes work, a fixed pipeline, or a graph of handoffs. The orchestrator manages

routing, shared state, retries, termination, and observability. Example: a Router agent sends a question to a SQL agent (Snowflake), a RAG agent (documents), or a Report agent.

Code

```
# Minimal hand-rolled orchestrator (LangGraph version in Card 16)
agents = {
    "sql":  lambda q: run_snowflake_agent(q),
    "docs": lambda q: rag_answer(q, retriever),
}

def route(q):
    r = client.messages.create(model="claude-haiku-5-5", max_tokens=5,
temperature=0,
        system="Reply with exactly one word: sql (metrics, numbers, tables)
or docs (policies, how-to).",
        messages=[{"role": "user", "content": q}])
    return r.content[0].text.strip().lower()

def orchestrate(q):
    return agents.get(route(q), agents["docs"])(q)
```

Card 15. When would you use a multi-agent architecture instead of one agent?

Answer: Use multiple agents when: tools or instructions are so numerous that one prompt degrades tool selection; subtasks need different models, permissions, or data access; subtasks can run in **parallel**; you want independent review (writer plus critic); or teams need to own and test components separately. **Stay with one agent** when the task is simple, latency and cost matter most, or coordination overhead would exceed the benefit. Multi-agent adds failure modes (handoff errors, loops, harder debugging), so start simple and split only when evals show a need.

Code

```
# Rule of thumb as code: a single agent with a tool list first
from langgraph.prebuilt import create_react_agent
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model="claude-sonnet-5-5")

single = create_react_agent(llm, tools=[search_docs, query_snowflake])
out = single.invoke({"messages": [("user", "Why did Q3 revenue drop?")]})
# Split into multiple agents only if this fails evals (wrong tool choice,
context overload, too slow)
```

Card 16. How would you design a supervisor agent using LangGraph?

Answer: Model the system as a graph: a **supervisor node** decides the next worker (or FINISH), worker nodes do the work and return to the supervisor, and a shared **state** carries messages and results. Use conditional edges for routing, a step cap to prevent loops, structured output for routing decisions, and a checkpointer for persistence.

Code

```
from typing import TypedDict, Annotated, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_anthropic import ChatAnthropic
from pydantic import BaseModel

llm = ChatAnthropic(model="claude-sonnet-5-5")

class State(TypedDict):
    messages: Annotated[list, add_messages]
    next: str
    steps: int

class Route(BaseModel):
    next: Literal["rag_agent", "sql_agent", "FINISH"]

def supervisor(state: State):
    decision = llm.with_structured_output(Route).invoke(
        [("system", "Pick the next worker: rag_agent (documents), sql_agent
(Snowflake metrics), or FINISH when the user's question is answered.")]
        + state["messages"])
    nxt = "FINISH" if state["steps"] >= 6 else decision.next   # loop guard
    return {"next": nxt, "steps": state["steps"] + 1}

def rag_agent(state: State):
    ans = rag_answer(state["messages"][-1].content, retriever)
    return {"messages": [("assistant", f"[rag] {ans}")]}

def sql_agent(state: State):
    return {"messages": [("assistant", f"[sql]
{run_snowflake_agent(state['messages'][-1].content)}")]}

g = StateGraph(State)
g.add_node("supervisor", supervisor)
g.add_node("rag_agent", rag_agent)
g.add_node("sql_agent", sql_agent)
g.add_edge(START, "supervisor")
g.add_conditional_edges("supervisor", lambda s: s["next"],
```

```
    {"rag_agent": "rag_agent", "sql_agent": "sql_agent", "FINISH": END})
g.add_edge("rag_agent", "supervisor")
g.add_edge("sql_agent", "supervisor")
app = g.compile()

print(app.invoke({"messages": [("user", "Summarize refund policy and show
last month's refund count")], "steps": 0}))
```

Try it: Visualize with `app.get_graph().draw_mermaid()`. Official tutorials: the LangGraph docs (langchain-ai.github.io/langgraph).

Part C: Performance, workflows, and reliability

Card 17. Rate limiting, concurrency, and performance

Answer: LLM providers enforce requests-per-minute and tokens-per-minute limits. Control them with: a **semaphore** to cap concurrent calls, **token-bucket** rate limiting, **retries with exponential backoff and jitter** on 429 and 5xx, queues for burst absorption, per-tenant quotas so one user can't starve others, and batching for offline jobs (embedding, evals). Watch p50 and p95 latency, queue depth, and 429 rate.

Code

```
import asyncio, anthropic
from tenacity import retry, wait_random_exponential, stop_after_attempt,
retry_if_exception_type

aclient = anthropic.AsyncAnthropic()
sem = asyncio.Semaphore(8)            # max 8 in-flight LLM calls

@retry(wait=wait_random_exponential(min=1, max=30),
stop=stop_after_attempt(5),
       retry=retry_if_exception_type((anthropic.RateLimitError,
anthropic.APIStatusError)))
async def call_llm(prompt):
    async with sem:
        r = await aclient.messages.create(model="claude-sonnet-5-5",
max_tokens=400,
                messages=[{"role": "user", "content": prompt}])
        return r.content[0].text

async def main(prompts):
    return await asyncio.gather(*(call_llm(p) for p in prompts))
```

Card 18. How do you optimize latency in a RAG pipeline?

Answer: Measure per stage first (embed, search, rerank, LLM first token, total). Then: **stream** the answer; run retrieval steps (vector plus keyword) in parallel; cache embeddings, retrieval results, and full answers (semantic cache); rerank only 20 to 30 candidates with a small model; trim context; use a smaller model for routing and easy questions; pre-warm connections; use ANN indexes with filters; keep infrastructure close together (same region); and avoid sequential LLM calls where one will do.

Code

```
import asyncio, time

async def timed(name, coro, log):
    t = time.perf_counter(); r = await coro
    log[name] = round(time.perf_counter() - t, 3); return r

async def retrieve(q, log):
    vec_hits, kw_hits = await asyncio.gather(          # parallel hybrid
search
        timed("vector", vector_search(q), log),
        timed("bm25", keyword_search(q), log))
    return rerank(q, merge(vec_hits, kw_hits)[:30], top=5)

# Streaming the answer cuts perceived latency
with client.messages.stream(model="claude-sonnet-5-5", max_tokens=600,
        messages=[{"role": "user", "content": prompt}]) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

Card 19. Explain the Saga pattern in an AI-driven business workflow.

Answer: A Saga splits a long business transaction across several services into local steps, each with a **compensating action** that undoes it if a later step fails, since a distributed transaction can't be rolled back atomically. In an AI workflow (e.g., an agent processes a refund: verify order, issue refund, update CRM, email customer), if "update CRM" fails you run compensations in reverse (reverse the refund, release the case). Steps must be **idempotent**, and state is persisted so the saga can resume after a crash. Keep a human approval step before irreversible actions.

Code

```
class Saga:
    def __init__(self): self.done = []
    def run(self, steps):
        try:
            for do, undo in steps:
                do(); self.done.append(undo)
        except Exception as e:
            for undo in reversed(self.done):
                try: undo()
                except Exception: log_for_manual_review(undo)
            raise

steps = [
    (lambda: reserve_credit(order_id),  lambda: release_credit(order_id)),
    (lambda: issue_refund(order_id),    lambda: reverse_refund(order_id)),
    (lambda: update_crm(order_id),      lambda: revert_crm(order_id)),
    (lambda: email_customer(order_id),  lambda:
send_correction_email(order_id)),
]
Saga().run(steps)
```

Try it: In LangGraph, model each step as a node and persist progress with a checkpointer; on failure route to a "compensate" node. AWS Step Functions also supports saga patterns natively with Catch blocks.

Card 20. What are the top three production challenges?

Answer:

1. **Quality and hallucination:** retrieval misses, stale documents, and unfaithful answers. Mitigate with evals in CI, monitoring, feedback loops, and abstention.
2. **Cost and latency:** token growth, slow multi-step agents, rate limits. Mitigate with caching, routing to smaller models, compression, and streaming.
3. **Security, governance, and reliability:** access control on retrieved data, prompt injection, PII leakage, provider outages, and non-deterministic agent loops. Mitigate with ACL filtering at retrieval time, input and output guardrails, audit logs, fallbacks, and step limits.

(Observability and evaluation across all three is the common thread.)

Code

```
# A tiny trace log that supports all three: quality, cost, reliability
import time, json, uuid
def traced(fn):
    def wrap(*a, **k):
        t = time.time(); rid = str(uuid.uuid4())[:8]
        try:
            out = fn(*a, **k); status = "ok"
            return out
        except Exception as e:
            status = f"error:{type(e).__name__}"; raise
        finally:
            print(json.dumps({"run": rid, "fn": fn.__name__, "sec":
round(time.time()-t, 2), "status": status}))
    return wrap
```

Try it: For real tracing use LangSmith or OpenTelemetry with CloudWatch.

Card 21. What happens when the LLM provider becomes unavailable?

Answer: Without planning, requests fail and users see errors. With resilience: retry transient errors with backoff; use a **circuit breaker** to stop hammering a failing provider; **fail over** to a secondary model or provider (for example, a second region or Amazon Bedrock); serve cached answers or degrade gracefully (show retrieved passages without a generated summary); queue non-urgent work for later; and alert on-call. Test this with chaos drills.

Code

```
import time

class CircuitBreaker:
    def __init__(self, max_fail=3, reset_sec=30):
        self.fails, self.max_fail, self.reset, self.opened = 0, max_fail,
reset_sec, 0
    def allow(self): return self.fails < self.max_fail or time.time() -
self.opened > self.reset
    def ok(self): self.fails = 0
    def bad(self):
        self.fails += 1
        if self.fails >= self.max_fail: self.opened = time.time()

primary, backup = CircuitBreaker(), CircuitBreaker()

def generate(prompt, context):
    for name, brk, fn in [("primary", primary, call_anthropic), ("backup",
backup, call_bedrock)]:
```

```
        if brk.allow():
            try: r = fn(prompt); brk.ok(); return r
            except Exception: brk.bad()
    # last resort: degrade gracefully
    return "The assistant is temporarily unavailable. Here are the most
relevant passages:\n" + context[:1500]
```

Card 22. How do you make the application fault-tolerant?

Answer: Design for failure at every layer: stateless services behind a load balancer across multiple AZs; **durable queues** (SQS) with dead-letter queues; **idempotent** operations with idempotency keys so retries are safe; **checkpointed** agent state so workflows resume; timeouts everywhere; retries with backoff and jitter; circuit breakers and fallbacks; health checks with auto-restart and autoscaling; backups of the vector store and state; and runbooks plus alarms. Define RTO and RPO targets and test recovery.

Code

```
import boto3, json, hashlib
sqs = boto3.client("sqs")
ddb = boto3.resource("dynamodb").Table("processed_requests")

def handle(msg):
    key = hashlib.sha256(msg["Body"].encode()).hexdigest()
    try:   # idempotency: skip if already processed
        ddb.put_item(Item={"id": key},
ConditionExpression="attribute_not_exists(id)")
    except ddb.meta.client.exceptions.ConditionalCheckFailedException:
        return
    run_agent(json.loads(msg["Body"]))

while True:
    for m in sqs.receive_message(QueueUrl=QUEUE_URL, WaitTimeSeconds=20,
MaxNumberOfMessages=5).get("Messages", []):
        handle(m)
        sqs.delete_message(QueueUrl=QUEUE_URL,
ReceiptHandle=m["ReceiptHandle"])
# Configure a redrive policy so messages failing N times go to a dead-letter
queue
```

Card 23. How would you deploy a RAG or multi-agent system on AWS?

Answer: A common architecture:

- **Edge:** Route 53, CloudFront, WAF, API Gateway or ALB with Cognito for auth.

- **Compute:** containerized FastAPI service on ECS Fargate (or EKS, or Lambda for light workloads), autoscaled across multiple AZs.
- **LLM:** Amazon Bedrock (Claude models) or the provider API via NAT or PrivateLink.
- **Data:** Snowflake for governed enterprise data and vectors (or OpenSearch or pgvector on Aurora); S3 for raw documents; DynamoDB or Postgres for agent state and checkpoints.
- **Async work:** SQS plus worker tasks for ingestion and long agent runs; Step Functions for saga-style workflows; EventBridge for schedules (dbt runs).
- **Ops:** Secrets Manager for keys, CloudWatch and X-Ray for logs, metrics, and traces, ECR plus CodePipeline or GitHub Actions for CI/CD, IaC with CDK or Terraform.

Code

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt . && RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:api", "--host", "0.0.0.0", "--port", "8080"]
```

```
# app.py
from fastapi import FastAPI
from pydantic import BaseModel
api = FastAPI()
class Ask(BaseModel): question: str; thread_id: str

@api.get("/health")
def health(): return {"ok": True}          # ALB health check

@api.post("/ask")
def ask(body: Ask):
    cfg = {"configurable": {"thread_id": body.thread_id}}
    out = app.invoke({"messages": [("user", body.question)], "steps": 0},
cfg)   # LangGraph app from Card 16
    return {"answer": out["messages"][-1].content}
```

```
# CDK sketch: pip install aws-cdk-lib
from aws_cdk import App, Stack, aws_ecs_patterns as ecsp, aws_ecs as ecs
class RagStack(Stack):
    def __init__(self, scope, id, **kw):
        super().__init__(scope, id, **kw)
        ecsp.ApplicationLoadBalancedFargateService(self, "Svc",
desired_count=2, cpu=1024, memory_limit_mib=2048,
            task_image_options=ecsp.ApplicationLoadBalancedTaskImageOptions(
                image=ecs.ContainerImage.from_asset("."),
container_port=8080))
App().synth()
```

Try it: Free tier is limited, so start locally with `docker run`, then `cdk deploy` to a sandbox account. Set a billing alarm first.

Card 24. How do you manage state in LangGraph?

Answer: State is a typed schema (TypedDict or Pydantic) shared by all nodes. Each node returns a **partial update**, and **reducers** (like `add_messages` or `operator.add`) define how updates merge. Short-term state is persisted per **thread** with a **checkpointer** (in-memory for dev, SQLite or Postgres in production), which enables resuming after failures, multi-turn memory, human-in-the-loop interrupts, and time travel. Long-term, cross-thread memory goes in a **Store**. Keep state small: store references (ids) to large objects, not the objects.

Code

```
import operator
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver   # prod: PostgresSaver
(pip install langgraph-checkpoint-postgres)

class State(TypedDict):
    messages: Annotated[list, add_messages]     # appended
    sources: Annotated[list, operator.add]      # appended
    answer: str                                 # overwritten

def retrieve_node(s: State):
    hits = retriever(s["messages"][-1].content, k=5)
    return {"sources": [h["text"] for h in hits]}

def answer_node(s: State):
    return {"answer": "...", "messages": [("assistant", "...")]}
```

```
g = StateGraph(State)
g.add_node("retrieve", retrieve_node); g.add_node("answer", answer_node)
g.add_edge(START, "retrieve"); g.add_edge("retrieve", "answer");
g.add_edge("answer", END)
app = g.compile(checkpointer=MemorySaver())

cfg = {"configurable": {"thread_id": "user-42"}}
app.invoke({"messages": [("user", "What is the refund window?")]}, cfg)
app.invoke({"messages": [("user", "And for enterprise plans?")]}, cfg)   #
remembers turn 1
print(app.get_state(cfg).values)
```

Try it: Add `interrupt_before=["answer"]` to `compile()` to pause for human approval, then resume with `app.invoke(None, cfg)`.

Suggested study and build order

1. Run Cards 3, 6, 7 locally (chunk, embed, FAISS) and measure Recall@5 (Card 13).
2. Move chunks into Snowflake with dbt, and swap in Cortex embeddings (Cards 2, 4).
3. Add reranking, abstention, and verification (Cards 5, 9, 10).
4. Wrap in LangGraph with a supervisor (Card 16) and checkpointer (Card 24).
5. Harden and deploy (Cards 17 to 23).